

Billy (BK) Rios » All Your Google Docs are Belong To US...

Billy (BK) Rios » All Your Google Docs are Belong To US... - Author not stated, xs-sniper.com.

- Published: date not stated
- Original: <<http://xs-sniper.com/blog/2007/09/28/all-your-google-docs-are-belong-to-us/>>
- Preserved from: <http://xs-sniper.com/blog/2007/09/28/all-your-google-docs-are-belong-to-us/> (stored) on 2026-08-09
- Capture timestamp: 20160408035749
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Billy (BK) Rios » All Your Google Docs are Belong To US...

Friday, September 28th, 2007

All Your Google Docs are Belong To US...

It's been a rough week for Google Security... It seems like everyone had some Google vulnerability they wanted to disclose this week. You can see some of the various vulns [here](#), [here](#), and [here](#).

Well... the week isn't over YET! I'm actually disclosing this vulnerability because Google has already fixed it. Although I don't use Google Docs (because I'm a paranoid guy), I know a lot of people who do and I didn't want to put their docs at risk. Without further delay, the details...

This vulnerability allowed any Google Docs user to STEAL ARBITRARY DOCUMENTS from the Google Docs Server. The basis of the vulnerability stems from a simple Session Management issue. Once a user has logged into Google Docs and has created a document, they are presented with several options. Under the "Share" tab, the user has an option to "Email Collaborators"

[[[image: Google Docs Share Tab](http://xs-sniper.com/blog/wp-content/uploads/2007/09/share-tab.jpg)]](<http://xs-sniper.com/blog/wp-content/uploads/2007/09/share-tab.jpg>)

Once the user clicks the "Email Collaborators" link, the following HTTP GET request is made to docs.google.com:

GET /Dialogs/EmailDocument?DocID=<ANY DOC ID HERE> HTTP/1.1 <appropriate HTTP headers here>

If you changed the DocID value to another DocID, Google Docs WOULD NOT VALIDATE whether you should have access to that DocID. The title of the stolen document you requested will be

shown (as a javascript variable) in the HTTP 200 OK response that is returned. Once this step is completed, you can make a POST request to a Google Docs Server Side Script named MiscCommands. The POST request looks something like this:

```
*POST /MiscCommands HTTP/1.1 <appropriate HTTP headers here>
```

```
command=validate_address&docid=<ANY DOCID
```

```
HERE>&addr=gmail%40gmail.com&finis=true&POST_TOKEN=POSTTOKENVALUE*
```

If you changed the DocID in the POST request, the entire contents of that document would be emailed to the addresses specified in the “addr” parameter! I tested this against several friends Google Docs and it worked EVERYTIME!

This issue does stem on being able to predict the DocID for the document that you want to steal. At first glance, the DocID seems to be a fairly stout “random string”, but a little bit of analysis shows some interesting characteristics. It seems that the DocID is delimited by an “_” character. The characters preceding the underscore represent the Google Docs UserID. Each document uploaded to Google Docs by a particular user will have the same characters up to the underscore. Now... what about the characters after the underscore? Well... take a look at what happens when I generate 10 different DocIDs in rapid succession:

```
mydocsid_14ggbd48 mydocsid_15gt54pt mydocsid_16c44jws mydocsid_17cnnfw8  
mydocsid_18ggzpm7 mydocsid_19dczf6g mydocsid_20c8h7nx mydocsid_21czqc3h  
mydocsid_22d48w8j mydocsid_23f4hk9b mydocsid_24gdwfkz
```

Maybe the last set of characters isn't as “random” as we thought..... Throw in some DocID enumeration (which exists) and we may be on to something here... I've seen Session Management issues like this in MANY of the web applications I've assessed. If your hired gun (webapp pentester) looks at you funny when you ask if they are testing for Session Management issues, FIND A NEW ONE! There isn't a web app vulnerability scanner on the market that can detect this and Web App firewalls will not prevent this either! It takes an actual brain and some experience to find these types of issues!

In closing, I would like to give a shout to the Google Security Team. If you've ever dealt with the Google Security Team, you know that they take security seriously and they move fast.... VERY FAST. After giving them the details for a couple of Google vulnerabilities, it took Google ONE DAY to fix the issues and to deploy the fixes worldwide... Kudos to Chris and the GST.

Posted by xssniper | Filed in [Security](#), [Web Application Security](#)

Please leave a Comment

Name (required)

Mail (will not be published) (required)

Website

Your Comment